Genetic Programming With Knowledge Transfer and Guided Search for Uncertain Capacitated Arc Routing Problem

Mazhar Ansari Ardeh, *Graduate Student Member, IEEE*, Yi Mei, *Senior Member, IEEE*, and Mengjie Zhang, *Fellow, IEEE*

***Abstract*—The uncertain capacitated arc routing problem has many real-world applications in logistics domains. Genetic programming (GP) is a promising approach to training routing policies to make real-time decisions and handle uncertain events effectively. In the real world, there are various problem domains and no single routing policy can work effectively in all of them. Instead of training in isolation, we can leverage the relatedness between the problems and transfer knowledge from previously solved source problems to solve the target problem. The existing transfer methods are not effective enough due to the loss of diversity during the knowledge transfer. To increase the diversity of the transferred knowledge, in this article, we propose a novel GP method that removes phenotypic duplicates from the source individuals to initialize the target individuals. Furthermore, assuming that the transferred knowledge used in initialization already includes all the important knowledge explored for the source problem, it is more effective to explore new regions that have not been explored for the source problem. Therefore, we propose novel genetic operators that prohibit the search from revisiting the source individuals when solving the target problem. To speed up the revisit check, we propose to adapt a powerful hashing method for routing policies that greatly improves the efficiency of the genetic operators. Our experimental results show that the proposed method significantly outperforms the existing GP approaches with knowledge transfer in terms of both initial and final solution quality.**

***Index Terms*—Arc routing, genetic programming (GP), guided search, hyperheuristics, transfer optimization.**

I. Introduction

THE CAPACITATED arc routing problem (CARP) [1], [2] is an important combinatorial optimization problem with many real-world applications, such as snow removal [3], network monitoring service [4], [5], and solid waste collection [6], [7]. Briefly speaking, CARP aims to design the routes for a fleet of vehicles with limited capacity to serve a collection of edges (called *tasks*) in a graph with minimal travel costs.

The uncertain CARP (UCARP) incorporates the uncertainties in many real-world scenarios [8], [9]. For example, the actual demand of a task may be different from its expected value due to inaccurate measurements and unpredictable customer behavior. Similarly, the travel costs are stochastic due to the uncertain traffic conditions. Considering these factors, Mei *et al.* [8] introduced four stochastic variables into CARP to express the environmental uncertainties.

CARP is NP-hard [1], [2] and UCARP inherits this property [8] as well. The presence of uncertainty adds to the difficulty of solving it. This is because adjusting the solution in real-time is difficult when the environment is different from expected. Due to these challenges, the exact and traditional solution optimization methods cannot perform well for UCARP [10]. On the other hand, equipping vehicles with *routing polices* provides a more effective and flexible alternative. A policy allows vehicles to make real-time routing decisions (e.g., the next task to serve) based on the up-to-date environmental information. Manually designing routing policies requires extensive domain expertise and is usually very time consuming. Genetic programming (GP) has been successfully applied to automatically evolving effective routing policies [10].

In the real world, there are many problem domains, such as snow removal and waste collection, that can be modeled as UCARP. These problem domains have the same problem model but different parameter values, such as the graph topology, variable distributions, and number of vehicles. No single routing policy can work well in all different problem domains.

Training a routing policy for each problem independently can be ineffective and cumbersome. On the other hand, the understanding that problems rarely exist in isolation [11], [12] leads to the development of *transfer optimization* methods that extract and reuse of knowledge from previously solved problems to boost the process of solving new but *related* problems [12].

A. Research Motivations

Transfer optimization has been proven to be effective for various problems [12]. However, the existing GP transfer

Manuscript received 9 February 2021; revised 5 June 2021 and 21 August 2021; accepted 2 November 2021. Date of publication 19 November 2021; date of current version 1 August 2022. This work was supported in part by the Marsden Fund of New Zealand Government under Contract VUW1509 and Contract VUW1614; in part by the Science for Technological Innovation Challenge (SfTI) Fund under Grant E3603/2903; and in part by the Ministry of Business Innovation & Employment (MBIE) Strategic Science Investment Fund (SSIF) Fund under Contract VUW RTVU1914. *(Corresponding author: Mazhar Ansari Ardeh.)*

The authors are with the Evolutionary Computation Research Group, School of Engineering and Computer Science, Victoria University of Wellington, Wellington 6140, New Zealand (e-mail: mazhar.ansariardeh@ecs.vuw.ac.nz; yi.mei@ecs.vuw.ac.nz; mengjie.zhang@ecs.vuw.ac.nz).

This article has supplementary material provided by the authors and color versions of one or more figures available at https://doi.org/10.1109/TEVC.2021.3129278.

Digital Object Identifier 10.1109/TEVC.2021.3129278

1089-778X © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

optimization methods have two major limitations. First, they typically transfer the individuals from the final population of the source problem which may have converged to local optima and contain redundancies and duplicates. This becomes more serious when many GP individuals have different genotypes (i.e., tree structures) but the same phenotypic behavior (routing behavior). The duplicated genetic materials in the transferred individuals can mislead the search for solving the target problem into poor local optima. Previous studies have shown such negative impact for UCARP [13]–[15].

The second limitation is that the existing methods mainly focus on high-quality source solutions, ignoring the low-quality ones. Although there are cases in which low-quality solutions are also considered [11], [16], [17], this is usually done in a limited fashion with the purpose of increasing the diversity in the pool and handling the situations when the source and target problems are not as related as expected [11]. Based on the assumption that the source and target problems are related and their fitness landscapes share similarities, the high (low)-quality solutions of the source problem tend to have high (low) fitness in the target problem. This knowledge about the search space of the source problem is important and can be beneficial in the target problem. It is even more important to consider a wider range of individuals from the source problem when the pool of good individuals is limited due to the lack of diversity [13].

B. Research Contributions

To address the above issues, we propose a novel GP with knowledge transfer and guided search (*GPKGS*). Our algorithm has two knowledge-guided components: 1) a new guided initialization and 2) newly proposed guided crossover and mutation operators. The contributions of this work are as follows.

1) Develop a new GP initialization method, which identifies the phenotypic duplicates from the source individuals and removes them before transferring to the target initial population.
2) Develop novel guided crossover and mutation operators that take advantage of all the source individuals to prevent the search from revisiting the regions that have been explored for the source problem.
3) Adapt a fast hashing algorithm for routing policies to speed up the comparisons to the large number of source individuals during the revisit check, which can greatly improve the efficiency of the guided crossover and mutation operators.

C. Organization

The remainder of this article is organized as follows. In Section II, a background review is given. In Section III, we describe the proposed *GPKGS* algorithm. Section IV presents the experiment design and the discussions of the obtained results. Finally, we conclude the article in Section V.

II. Background

A. Uncertain Capacitated Arc Routing Problem

In UCARP, a set of vehicles is destined to serve a collection of tasks in an environment ξ. In this problem, the environment is represented as a graph $G = (V, E)$. The set of nodes V models the locations of interest in the environment and the set of edges E represents the routes. All vehicles are stationed initially at a depot node $v_0 \in V$. Each vehicle has a finite capacity Q. In UCARP, tasks are spread over edges. The amount of work, $d_\xi(e)$, for the edge $e \in E$ is called its *demand*. Edges with nonzero demands are called *tasks*. Two cost values are associated with an edge: 1) a *serving cost* $sc(e) > 0$ and 2) a *deadheading cost* $dc_\xi(e) > 0$. In UCARP, the presence of routes, the presence of tasks, the demand, and deadheading costs have stochastic natures that are designed to reflect the uncertainties of real-world environments [8], [10]. Due to these uncertainties, serving tasks may encounter failures. A *route failure* happens when a vehicle arrives at a task whose demand is greater than its capacity. An *edge failure* happens if vehicles perceive the deadheading cost of their selected task to be infinite (e.g., the road is closed due to roadwork). In the case of a route failure, the vehicle needs to return to the depot and replenish its capacity. For edge failures, the vehicle must find a route around the blocked road.

The goal in solving UCARP is to plan a set of routes for the vehicles that serves all tasks with the minimum total cost. However, due to the uncertain nature of the problem, the objective is to find a robust solution with the best expected quality for all possible environments Ξ, rather than a single global solution. Hence, if S_ξ is the UCARP solution for the environment $\xi \in \Xi$ with the total cost of $tc(S_\xi)$, then the goal is to find a solution S with the minimum expected total cost

$$R(S) = E\big[tc\big(S_\xi\big)|\xi\big]. \tag{1}$$

Any UCARP solution must respect the following constraints: 1) the *depot* node must be the first and last node of all routes; 2) each task must be served exactly once except when a route failure occurs in which case the failed task is served twice; and 3) vehicles cannot serve when their capacity is exhausted. In this case, they need to return to depot and replenish their capacity.

Broadly speaking, the methods for solving UCARP can be categorized into *proactive* and *reactive* approaches. In the *proactive* category, usually a metaheuristic algorithm is utilized to find a robust solution for a UCARP instance. The works in [8], [18], and [19] are good examples that belong to this category.

Because the proactive methods ignore the uncertainty of environment during the optimization phase, their results do not possess desirable flexibility for real-world scenarios. The *reactive* category of methods address this shortcoming by equipping vehicles with routing policies which let them build their routes in real time based on the most recent environmental information. Routing policies are, in essence, priority functions. Whenever a vehicle becomes idle, it uses the routing policy to calculate the priority of the available tasks and selects the most prior task to serve next. Since the vehicles can use

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

the most recent information about the current environment, this approach can depict great flexibility compared to the proactive methods [10].

Liu and Mei [10] were the first to argue that since UCARP routing policies are heuristics in nature, GP can be used as a hyperheuristic for evolving them. Accordingly, they designed a GP hyperheuristic (GPHH) approach to evolve routing policies for the single-vehicle version of UCARP and demonstrated that the automatically evolved routing policies can outperform manually designed ones. While Liu and Mei [10] considered the single-vehicle version of UCARP, Mei and Zhang [20] extended their work for multiple vehicles. They also noted that the efficiency of vehicles can be improved if inaccessible tasks are filtered out before calculating the priority of unserved tasks. MacLachlan *et al.* [21] showed that the performance of vehicles will increase if they are allowed to collaborate. The routing policies evolved by GPHH are usually difficult to interpret. Hence, Wang *et al.* [22], [23] evolved smaller but more interpretable routing policies. In a later work, Wang *et al.* [24] partitioned the GP population into distinct niches based on the fitness value, selected the smallest individual of each niche into an archive, and modified the breeding operators to utilize the archive for creating smaller and more interpretable policies.

B. Genetic Programming Hyper Heuristics

GP is an evolutionary algorithm (EA) that is designed to evolve computer programs automatically [25], [26]. GP has found applications in a vast variety of problems [27]–[30]. GPHH is a special case of GP when it is utilized for searching the space of heuristic functions [31]–[33].

In the context of dynamic and uncertain environments like UCARP [34], a heuristic function $\rho : \mathbf{\Xi} \rightarrow \mathbb{R}$ is a mathematical expression that maps a state of the environment $\mathbf{\Xi}$ to a real number in $\mathbb{R}$. The definition of the environment state is problem-dependent. For example, in UCARP, the environment state contains the information about the status of vehicles and uncompleted tasks.

GP can search for the routing policies of UCARP. A standard GP [26] has four main steps: 1) creating an initial random population of heuristics; 2) evaluating the fitness of the population; 3) evolving the population with the crossover, mutation and reproduction operators; and 4) repeating steps 3 and 4 until some stopping criteria is met.

Apart from UCARP, GPHH has also been successfully applied to other uncertain and dynamic problems. Tan *et al.* [35] developed a novel cooperative GPHH approach for on-line resource allocation and Zhang *et al.* [34] evolved heuristics for dynamic flexible job-shop scheduling with GPHH. The GPHH technique has also been applied successfully to online bin packing [31].

C. Evolutionary Transfer Optimization

In the evolutionary computation realm, *transfer optimization* is the general term for the algorithms that aim to extract knowledge from solved source optimization problems to improve the efficiency and/or effectiveness for solving the target optimization problem [12].

Most of the early transfer optimization efforts were focused on *injection of genetic materials* from a solved source problem into the population of the algorithm that solves a target problem. Cunningham and Smyth [36] proposed starting an optimization process for a new problem with solutions that were either transferred as whole or were constructed from components of several solutions. Louis and McDonnell [11] proposed to inject periodically into the population the transferred solutions that are most similar to the best solutions of the current population. Taylor *et al.* [37] performed knowledge transfer through initializing genetic algorithm (GA) with the final population of the source problem, Koçer and Arslan [16] proposed a *GATL* method that selects the best and median individuals of each generation of the source problem to initialize the GA for the target problem. Iqbal *et al.* [38] considered the subtrees of solved solutions of the source problem as *code fragments* that have potentially important genetic materials. Their algorithm, *TLGPC*, reuses these codes during initialization and mutation phases. Limiting the size of subtrees to two, Iqbal *et al.* [39] extracted important subtrees from the found source solutions and converted them to GP terminals to solve the target problem. Similar schemes for reusing the genetic materials were explored in [13], [15], [17], [40], and [41].

Transfer optimization can also be achieved through learning a model of the good solutions of the source problem. In these *model-based* approaches, the model is then used for biasing the search process when solving the target problem [42]. Feng *et al.* [43], [44] defined the transferable model as positive semidefinite matrices to map problem instances to their solutions. Feng *et al.* [45] performed knowledge transfer through a single-layer denoising autoencoder. A similar idea proved to be useful for knowledge sharing in multitask scenarios [46]. Ardeh *et al.* [14] learned feature importance of GP terminals for solving UCARP from a solved source problem and used it for a target problem. Zhou *et al.* [47] captured the structured knowledge from the optimized vehicle routing problem (VRP) and reused it for biasing the search when the problem changes.

Probabilistic models are also popular for capturing and transferring knowledge between problems. For example, it is shown that previously learned/evolved probabilistic models can be used as transferable knowledge. In order to achieve this, a similarity metric can be designed on problem variables to indicate the strength of dependencies between them. The metric can then be used for collecting statistics on the transferred models and biasing the search process when similar problems are solved in the future [48]–[51]. Ardeh *et al.* [52] captured and transferred the probability distribution of good UCARP solutions of the source problem as a probabilistic prototype tree.

Another approach to transfer optimization is to treat it as a dynamic optimization problem (DOP) [53] and adapt/apply the algorithms for solving DOPs accordingly [12]. Ardeh *et al.* [54] proposed *DDGP*, which reused the final population of the source problem for initializing the target problem but set the mutation rate to a high value and decreased it

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

gradually throughout the evolution. This approach, generally referred to as hypermutation in the literature of DOP [53], proved to be effective but they concluded that stronger methods are required for handling the lack of diversity issue in the knowledge source. The work in [52] can also be considered a DOP memory-based approach to knowledge transfer.

In contrast with GA, the number of studies in the context of GP is very limited [55]. Dinh *et al.* [17] proposed three methods for transfer optimization. In their *FullTree-k* method, *k*% of GP population is initialized with the individuals of the final population of the source problem. The *SubTree-k* algorithm extracts the immediate subtrees of the root of the individuals in the final population of the source problem, and initializes *k*% of the target population by randomly selecting the subtrees from the pool. The *BestGen* method forms the pool using the best individuals in each generation rather than the final generation only. O'Neill *et al.* [40] and Ardeh *et al.* [13] proposed more intelligent methods to select the (sub-)trees based on their frequencies rather than purely randomly. Al-Helali *et al.* [56] proposed a multitree GP method with new genetic operators based on transferred knowledge and utilized it for solving symbolic regression problems with incomplete data. Muñoz *et al.* [55] have given a comprehensive review of transfer optimization methods for GP in the context of constructive induction.

The transfer of useful knowledge is not specific to transfer optimization but is also an important concept in the context of evolutionary multitask (EMT) learning [57], [58]. Feng *et al.* [59] proposed an explicit EMT that utilized autoencoders during knowledge transfer for solving CARP. Zhou *et al.* [60] performed knowledge transfer that is self-adaptive based on the information obtained during the search process. Feng *et al.* [61] learned customer representation as transferable knowledge for solving VRPs. Feng *et al.* [62] also proposed a novel EMT algorithm to solve a generalized VRP with occasional drivers. Wei and Zhong [63] proposed an EMT-based classification method using Gene Expression Programming with several knowledge transfer strategies between tasks.

Most existing works for UCARP do not consider the duplicated phenotypic behaviors in the source individuals, causing a lack of diversity in the transferred knowledge. Additionally, they mainly reuse the transferred knowledge in initialization, but not during the search process. In this article, to address the first issue, we propose a new initialization with knowledge transfer, which removes duplicates based on phenotypic characterizations when initializing target individuals. To address the second issue, we develop novel guided crossover and mutation operators that use the source knowledge to prevent GP from revisiting the individuals that have been explored for the source problem. Furthermore, we adapt a powerful hashing algorithm for routing policies that greatly improves the efficiency of the guided the crossover and mutation operators.

III. Proposed Algorithm

A. Overall Framework

The overall framework of our proposed *GPKGS* is depicted in Fig. 1. *GPKGS* has two inputs: 1) a target problem to solve and 2) the knowledge transferred from the source problem(s). Here, we consider all the individuals evaluated by GP for solving the source problem, $\mathcal{S}$, as the transferred knowledge.

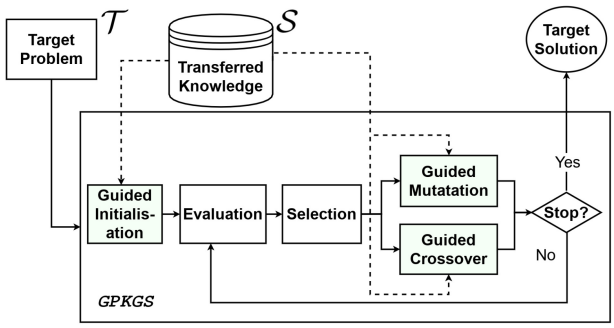


Fig. 1. *GPKGS* framework.

Compared with the existing GP transfer optimization methods, the proposed *GPKGS* contains the following novel contributions. First, *GPKGS* has a novel initialization that removes the phenotypic duplicates from the source individuals before transferring them. This has substantially increased the diversity of the transferred knowledge. Second, *GPKGS* contains novel guided crossover and mutation operators. In contrast to most existing methods that use the transferred knowledge to generate (better) individuals, the guided crossover and mutation operators use the transferred knowledge to avoid revisiting the source individuals and, thus, encourage exploring new regions. The guided crossover and mutation operators are assisted by a powerful hashing algorithm that greatly improves their computational efficiency.

Algorithm 1 presents the pseudocode of *GPKGS*. It receives the transferred knowledge $\mathcal{S}$ from a previously solved source problem and a target problem $\mathcal{T}$ as inputs and returns the best solution for $\mathcal{T}$. The algorithm starts by creating a hash table $\mathbf{\Psi}$ (line 3), which contains a summary of $\mathcal{S}$ for a fast query of its content. The details about $\mathbf{\Psi}$ will be given in Section III-B2. Then, the guided initialization (as shown in Algorithm 2) is used to create the initial GP population. After this, the population is evolved for *MaxGen* number of generations. In each generation, the fitness of each individual in the population is evaluated (line 7) and the best individual is updated (line 8). Then, a new population is bred by the proposed `GuidedCrossover` (Algorithm 7), `GuidedMutation` (Algorithm 8), reproduction, and elitism [25] (lines 10–27).

Compared with the existing GP transfer optimization methods, the proposed *GPKGS* contains the following novel contributions. First, *GPKGS* has a novel initialization that removes the phenotypic duplicates from the source individuals before transferring them. This can substantially increase the diversity of the transferred knowledge. Second, *GPKGS* contains novel guided crossover and mutation operators. In contrast to most existing methods that use the transferred knowledge to generate (better) individuals, the guided crossover and mutation operators use the transferred knowledge to avoid revisiting the source individuals and, thus, encourage exploring new regions. The guided crossover and mutation operators are assisted by a powerful hashing algorithm that greatly improves their computational efficiency.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Pseudocode for *GPKGS*

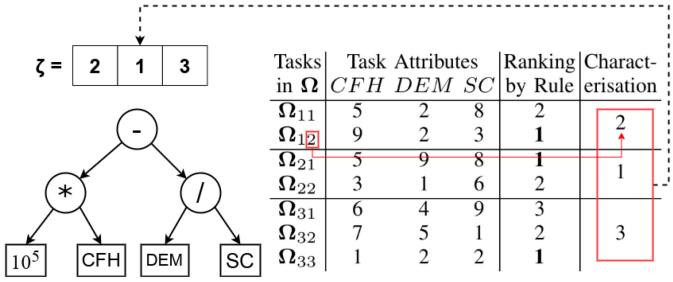
Input : κ, the percentage of the initial population to transfer from the source problem;
$\mathcal{S}$, all GPHH individuals discovered for solving source problem; $\mathcal{T}$, target problem to solve
Output: *Ind**, the best solution for target problem $\mathcal{T}$

```
1  begin
2      Ind* ← null
3      Ψ ← GenerateHashTable (S) (Algorithm 5)
4      Pop ← GuidedInitialisation (Ψ, κ) (Algorithm 2)
5      g ← 0
6      while g < MaxGen do
7          Evaluate (Pop)
8          Ind* ← SelectBest (Pop)
           // Create a new population
9          Pop' ← Elitism ()
10         while |Pop'| < |Pop| do
11             op ← SelectOperator ()
12             switch op do
13                 case 'Crossover' do
14                     ρ1, ρ2 ← TournamentSelect (Pop, 2)
                       // See Algorithm 7
15                     ρ'1, ρ'2 ←GuidedCrossover (Ψ, ρ1, ρ2)
16                     Pop' ← Pop' ∪ {ρ'1, ρ'2}
17                 end
18                 case 'Mutation' do
19                     ρ ← TournamentSelect (Pop, 1)
                       // See Algorithm 8
20                     ρ' ← GuidedMutation (Ψ, ρ)
21                     Pop' ← Pop' ∪ {ρ'}
22                 end
23                 case 'Reproduction' do
24                     ρ' ← Reproduction (Pop)
25                     Pop' ← Pop' ∪ {ρ'}
26                 end
27             end
28             g ← g + 1
29             Pop ← Pop'
30         end
31     end
32     return Ind*
33 end
```

Algorithm 2: `GuidedInitialisation`($\mathcal{S}$, Ψ, κ)

Input : $\mathcal{S}$, all GPHH individuals discovered for solving source problem; Ψ, the hashed summary of $\mathcal{S}$; κ, the percentage of the initial population to transfer from the source problem
Output: *Pop*, the initial GPHH population

```
1  Pop ← ∅
2  S' ← RemoveDuplicate (S) (Algorithm 3)
3  S' ← SortFitness (S') // from best to worst
4  Pop ← SelectBest (S', κ) // Select the best
5  while |Pop| < PopSize do
6      ind ← RandIndividual ()
7      if IsSeen (Ψ, ind) = false then
8          Pop ← Pop ∪ {ind}
9      end
10 end
11 return Pop
```



Tasks in Ω	Task Attributes CFH	DEM	SC	Ranking by Rule	Characterisation
Ω_{11}	5	2	8	2	2
Ω_{12}	9	2	3	**1**	
Ω_{21}	5	9	8	**1**	1
Ω_{22}	3	1	6	2	
Ω_{31}	6	4	9	3	
Ω_{32}	7	5	1	2	3
Ω_{33}	1	2	2	**1**	

Fig. 2. Tree representation of the $10^5 * CFH - DEM/SC$ path-scanning heuristic and its phenotypic characterization.

B. Representation and Fitness Evaluation

In our approach, we adopt the commonly-used tree representation for routing policies (e.g., Mei *et al.* [8]). As an example, Fig. 2 depicts the tree representation of the well-known path-scanning heuristic [64] as 10^5 * *CFH* - *DEM/SC*, where *CFH* indicates the cost from the vehicle's current location to the candidate task, *DEM* is the demand of the task, and *SC* is its serving cost (the coefficient 10^5 of *CFH* is a sufficiently large number so that smaller *CFH* is always preferred).

Equation (1) defines the fitness of a routing policy ρ but it cannot be utilized directly because we cannot enumerate all possible environments. Instead, the fitness value of a routing policy is defined based on a set of training instance samples [8]. For each instance sample, the routing policy ρ is applied to generate the solution, i.e., the set of routes that serve all the tasks. For this, all vehicles are initially assumed to be idle and stationed at the depot node and all the tasks are unserved. When a vehicle becomes idle, it considers all the remaining tasks, filters out the tasks it cannot serve, utilizes ρ to compute the priorities of the feasible tasks and selects the most prior task to serve next. If a routing or edge failure occurs, the failed vehicle replenishes its capacity via visiting the depot or finding a route around the blocked route. This process is repeated until all the tasks are served and all the vehicles are returned to the depot [8], [10]. Finally, the average total cost of the generated solutions is set as the fitness of ρ.

The newly developed guided initialization is described in Algorithm 2. First, the algorithm removes the duplicated trees from $\mathcal{S}$ (line 2). This is different from the existing GP transfer methods, which do not take into account the possible presence of redundancy in the transferred pool. Then, the unique individuals are sorted based on their fitness for the source problem (line 3), and $\kappa\%$ of GP population for the target problem is initialized with the best of them (line 4). The remaining of the population, considering the situation in which the number of unique individuals is smaller than $\kappa\%$, is filled with randomly-created individuals (lines 5–10) that are not explored in the source problem. Specifically, for adding each individual, we first create a new individual using the Ramped Half and Half approach [25]. Then, we scan the hash table Ψ and add the new individual into the population if it is not seen in Ψ.

1) Duplicate Removal: Algorithm 3 shows the `RemoveDuplicate` function. The function receives a set of individuals *P* as input, sets the fitness of the duplicates to infinity (essentially removes them), and returns a set of unique individuals.

In line 7, a distance measure Δ is utilized to determine if two individuals are identical or not. Since routing policies with different genotypic structures can have identical phenotypic behaviors [65], we choose the phenotypic characterization approach [65], [66] to measure the dissimilarity of routing policies.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: `RemoveDuplicate`(P)

```
Input: A set of individuals P
Output: A set of unique individuals P' ⊂ P
1  begin
2      P' ← ∅
3      P ← SortFitness (P)
4      for i = 1 → |P| do
5          if Fit (P[i] < ∞) then
6              for j = i + 1 → |P| do
7                  if Fit (P[j] < ∞) and Δ(P[i], P[j]) = 0 then
8                      Fit(P[j]) ← ∞
9                  end
10             end
11             P' ← P' ∪ {P[i]}
12         end
13     end
14     return P'
15 end
```

Algorithm 4: `Characterise`(ρ, Ω)

```
Input: A routing policy ρ; a set of decision situations Ω
Output: A numeric characteristic vector ζ
1  foreach Ω_i ∈ Ω do
2      r ← rank (ρ, Ω_i)
3      j ← index (r)
4      ζ_i ← j
5  end
6  return ζ
```

Algorithm 4 describes the phenotypic characterization process. The algorithm characterizes a routing policy ρ based on a set of decision situations Ω. A decision situation contains the state of environment when a vehicle is idle and needs to select an unserved task to serve next from the candidate task list. For each $\Omega_i \in \Omega$, it first calculates the priority of each candidate task and ranks them based on their priorities (line 2). Then, it obtains the index of the top-rank task in the candidate task list (line 3) in the set of unserved tasks in Ω_i. Fig. 2 depicts the phenotypic characterization process. It should be noted that the phenotypic characterization operation maps the set of routing policies to the $\mathbb{R}^d$ space for $d = |\Omega|$ and, hence, the Euclidean distance can be defined to measure the difference between two routing policies ρ and q

$$\Delta(\rho, q) = \left(\sum_{i=1}^{|\Omega|} (\zeta_{\rho i} - \zeta_{qi})^2 \right)^{1/2}. \tag{2}$$

In this direction, two policies ρ and q are considered the same if they select the same tasks for all decision situations (and hence $\Delta(\rho, q) = 0$). It should be noted that since we defined (2) based on task indices, when $\Delta(\rho, q) \neq 0$, the calculated distance is not accurate (its accuracy may vary depending on the number of same tasks that the policies select). However, in this work, this is not problematic because we are only interested in cases where $\Delta(\rho, q) = 0$.

2) Hashing the Transferred Knowledge $\mathcal{S}$: To search for an individual q in $\mathcal{S}$, a straightforward approach is to simply compare each $s \in \mathcal{S}$ with q based on the dissimilarity function Δ (2). This approach has a time complexity of $O(n)$ where $n = |\mathcal{S}|$, which can be very slow. To address this issue, we employ an approximate search technique used for solving the nearest neighbor search problems that can search much more efficiently in large datasets [67], [68]. Although there exist exact search methods for low dimensions (e.g., dimensionality of 2 or 3) [69], [70], for high dimensions, these methods do not perform better than the linear search [71]. Hence, approximate methods with better time complexities have been proposed that are almost as effective as the exact ones [72]–[75].

In this article, we employ the locality-sensitive hashing (LSH) technique [70] that has a sublinear dependence on data size. Indyk and Motwd [70] originally proposed the method for the d-dimensional binary Hamming space $H = \{0, 1\}^d$ and Datar *et al.* [69] extended LSH for the Euclidean space $\mathbb{R}^d$.

The key idea of LSH is to hash the items of a set S using several hashing functions, called *LSH functions* or *LSH family*, in a way that if two data items are more similar to each other, the probability of collision (having the same hash value) is higher. For $\mathbb{R}^d$, the LSH function can be of the form

$$h_{\boldsymbol{a},b}(\boldsymbol{o}) = \left\lfloor \frac{\boldsymbol{a} \cdot \boldsymbol{o} + b}{w} \right\rfloor. \tag{3}$$

This LSH function projects the vector $\boldsymbol{o} \in \mathbb{R}^d$ into buckets of width w by first projecting it along a random line identified by $\boldsymbol{a} \in \mathbb{R}^d$ with the inner product operation and then, shifting the projection by a random shift value $b \in \mathbb{R}$. Finally, the bucket that the projection $\boldsymbol{a} \cdot \boldsymbol{o}$ and, hence $\boldsymbol{a}$, falls into is located by applying the floor function. To reduce the probability that distant/dissimilar data items fall into the same bucket, a compound hash function is created by constructing λ hash functions via choosing a set of random lines $\boldsymbol{A}$ and shift values $\boldsymbol{B}$ and concatenating their results

$$h(\boldsymbol{o}) = \bigoplus_{\boldsymbol{a},b \in \boldsymbol{A},\boldsymbol{B}} h_{\boldsymbol{a},b}(\boldsymbol{o}) \tag{4}$$

where $\bigoplus$ represents the concatenation operator. To further decrease the chance of incorrect collisions, τ compound hash functions can be considered. Having a hash function, any dataset can be effectively hashed into τ hash tables [69], [70]. As a result, given a query point q, first its hash value is calculated and then, the hash tables are queried and all the items with the same hash value are compared to q to find the match [69], [70], [76], [77].

The phenotypic characterization in Algorithm 4 and the norm in (2) map routing policies to the $\mathbb{R}^d$ space. This allows us to utilize the LSH technique for efficient hashing and searching of the transferred archive $\mathcal{S}$ in a straightforward manner. Algorithms 5 and 6 present the process. First it is needed to hash the routing policies in $\mathcal{S}$ and create the set of hash tables $\boldsymbol{\Psi}$ with Algorithm 5. Then, Algorithm 6 is used to search for a policy q in $\boldsymbol{\Psi}$. Specifically, the hash value of q is calculated first (lines 2 and 3) and looked up in each hash table (line 5). If the hash value is in the table, the corresponding routing policies are acquired from the table. The retrieved set has a small size and by comparing the policies inside it against q (lines 6–10), it can be realized if q is in $\mathcal{S}$ or not.

Algorithm 5 hashes each $s \in \mathcal{S}$ for τ times and stores them in τ hash tables. As a result, the algorithm has $O(\tau n)$ time and space complexities. Algorithm 6 does not store any data and, hence, has a space complexity of $O(1)$. Datar *et al.* [69]

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5: Equation (4), `GenerateHashTable`($\mathcal{S}$)

Input: $\mathcal{S}$, history of all GPHH individuals discovered for solving source problem
Output: Ψ, a hash table of the policies in $\mathcal{S}$ that maps a hash value to the set of routing policies with the same value

```
1  begin
       // The table of hashed individuals in S
       // to return.
2      Ψ ← {}
3      for i = 1 → τ do
4          Ψ ← ∅
5          for s ∈ S do
6              h ← Hash (s) (Equation 4)
7              Ψ[h] ← Ψ[h] ∪ {s}
8          end
9          Ψ ← Ψ ∪ {Ψ}
10     end
11 end
12 return Ψ
```

Algorithm 6: `IsSeen`(Ψ, ρ)

Input: Ψ, history of all GPHH individuals discovered for solving source problem, hashed with Algorithm 5; An individual ρ to search for in Ψ
Output: **true** if ρ is in Ψ and, **false** otherwise

```
1  begin
2      ζ ← Characterise (ρ) (Algorithm 4)
3      h ← Hash (ζ)
4      for Ψ ∈ Ψ do
           // Get policies with the same hash value
5          S' ← Ψ[h]
6          for s ∈ S' do
7              if Δ(s, ρ) = 0 then
8                  return true
9              end
10         end
11     end
12     return false
13 end
```

proved that the time complexity of Algorithm 6 cannot be worst than $O(\tau n^{1/2} \log n)$, which is significantly better than the $O(n)$ time complexity of the linear search approach.

C. Guided Genetic Operators

To take better advantage of the transferred knowledge $\mathcal{S}$, the guided genetic operators consider the hash table Ψ, and retain only the generated offspring that are not seen in Ψ. This way, the search for the solutions of the target problem can focus more on the regions that are not explored when solving the source problem.

Algorithm 7 presents `GuidedCrossover`, the crossover operator of *GPKGS*. The algorithm takes two parents ρ_1 and ρ_2, and the hash table Ψ. After performing the standard pointwise crossover [25] and creating two offspring, the algorithm uses the `IsSeen` function (Section III-B2) to check if any of the offspring has been seen in ψ. An offspring is stored in the Γ set if it is new. The crossover operation is repeated to create new offspring until either two new offspring are found or a the maximum number of trials η is reached (lines 4–10). The parameter η controls the amount of effort that *GPKGS* exerts for finding new and unseen individuals. It should be noted

Algorithm 7: `GuidedCrossover`(Ψ, ρ_1, ρ_2)

Input: Ψ, a hashed history of all GPHH individuals discovered for solving source problem; Two individuals ρ_1 and ρ_2 to perform crossover on
Output: Two offsprings ρ_1' and ρ_2'

```
1  begin
       // Number of trials
2      try ← 0
3      Γ ← {}
4      while try ≤ η and |Γ| < 2 do
5          try ← try + 1
6          ρ1', ρ2' ← Crossover (ρ1, ρ2)
7          if IsSeen (Ψ, ρ1') = false then
8              Γ ← Γ ∪ {ρ1'}
9          end
10         if IsSeen (Ψ, ρ2') = false then
11             Γ ← Γ ∪ {ρ2'}
12         end
13     end
       // In case |Γ| < 2
14     Γ ← {ρ1', ρ2'}
15     return Γ[1], Γ[2]
16 end
```

Algorithm 8: `GuidedMutation`(Ψ, ρ)

Input: $\mathcal{S}$, history of all GPHH individuals discovered for solving source problem; an individual ρ to mutate
Output: A mutated individual ρ'

```
1  begin
       // Number of trials
2      try ← 1
3      ρ' ← Mutate (ρ)
4      found ← false
5      while try ≤ η do
6          ρ' ← Mutate (ρ)
7          try ← try + 1
8          if IsSeen (Ψ, ρ') = false then
9              found ← true
10             break
11         end
12     end
13     if found = false then
14         r ← rand (0, 1)
15         if r ≤ 0.5 then
16             ρ' ← randomly created individual
17         end
18     end
19     return ρ'
20 end
```

that for $\eta = 1$, `GuidedCrossover` reduces to the standard pointwise crossover. Finally, the Γ set is returned as the output of `GuidedCrossover`. If Γ contains less than two individuals, the most recently created individuals are returned as needed (lines 14 and 15).

Algorithm 8 shows the `GuidedMutation` operator. Given the parent individual ρ, it is first mutated with the standard mutation [25] (line 3 and 6) into ρ'. Then, ρ' is checked against Ψ, to find out if it has been seen previously or not (line 8). If ρ' is seen, then it is discarded and a new mutation is performed on ρ with the hope of finding a previously unseen individual. This process is repeated for preset number of η times (lines 5–12).

After multiple tries, the mutation operator may not yield a new individual. In this case, one option is to accept the mutation result, even if it is seen before. On the other hand, since the intention of the mutation is to introduce diversity

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TRANSFER SCENARIOS USED IN THE EXPERIMENTS

	Source Problem	Target Problem	r
1	Ugdb15 with 4 vehicles	Ugdb15dm1 with 4 vehicles	0.44
2	Uval4A with 2 vehicles	Ugdb17 with 3 vehicles	0.44
3	Uval6B with 5 vehicles	Ugdb15 with 3 vehicles	0.45
4	Ugdb17 with 5 vehicles	Ugdb11 with 5 vehicles	0.45
5	Uegl-e1-C with 8 vehicles	Ugdb13 with 5 vehicles	0.45
6	Uval6B with 3 vehicles	Ugdb20 with 3 vehicles	0.45
7	Uval8B with 3 vehicles	Ugdb23 with 9 vehicles	0.46
8	Ugdb21 with 5 vehicles	Ugdb5 with 4 vehicles	0.49
9	Ugdb4 with 4 vehicles	Ugdb4 with 2 vehicles	0.62
10	Ugdb4 with 4 vehicles	Ugdb4 with 6 vehicles	0.65
11	Ugdb4 with 4 vehicles	Ugdb4 with 3 vehicles	0.66
12	Ugdb1 with 5 vehicles	Ugdb1 with 3 vehicles	0.67
13	Ugdb6 with 5 vehicles	Ugdb7 with 6 vehicles	0.67
14	Ugdb3 with 5 vehicles	Ugdb3 with 3 vehicles	0.68
15	Ugdb6 with 5 vehicles	Ugdb6 with 3 vehicles	0.68
16	Ugdb7 with 5 vehicles	Ugdb1 with 6 vehicles	0.68
17	Ugdb3 with 5 vehicles	Ugdb3 with 7 vehicles	0.69
18	Ugdb7 with 5 vehicles	Ugdb7 with 3 vehicles	0.69
19	Ugdb1 with 5 vehicles	Ugdb1 with 7 vehicles	0.7
20	Ugdb5 with 6 vehicles	Ugdb5 with 4 vehicles	0.7
21	Ugdb1 with 5 vehicles	Ugdb2 with 7 vehicles	0.71
22	Ugdb5 with 6 vehicles	Ugdb5 with 8 vehicles	0.71
23	Ugdb21 with 6 vehicles	Ugdb21 with 4 vehicles	0.71
24	Ugdb4 with 4 vehicles	Ugdb4 with 5 vehicles	0.72
25	Ugdb6 with 5 vehicles	Ugdb6 with 7 vehicles	0.72
26	Ugdb7 with 5 vehicles	Ugdb7 with 7 vehicles	0.72
27	Ugdb7 with 5 vehicles	Ugdb7 with 4 vehicles	0.73
28	Ugdb2 with 6 vehicles	Ugdb2 with 4 vehicles	0.74
29	Ugdb2 with 6 vehicles	Ugdb6 with 6 vehicles	0.74
30	Ugdb6 with 5 vehicles	Ugdb6 with 4 vehicles	0.74
31	Ugdb3 with 5 vehicles	Ugdb3 with 6 vehicles	0.75
32	Ugdb21 with 6 vehicles	Ugdb21 with 5 vehicles	0.75
33	Ugdb2 with 6 vehicles	Ugdb2 with 8 vehicles	0.77
34	Ugdb1 with 5 vehicles	Ugdb1 with 6 vehicles	0.77
35	Ugdb2 with 6 vehicles	Ugdb2 with 5 vehicles	0.77
36	Ugdb6 with 5 vehicles	Ugdb6 with 6 vehicles	0.77
37	Ugdb18dm1 with 5 vehicles	Ugdb18dm2 with 5 vehicles	0.79
38	Ugdb2 with 6 vehicles	Ugdb2 with 7 vehicles	0.8
39	Ugdb23 with 10 vehicles	Ugdb23 with 8 vehicles	0.8
40	Ugdb23 with 10 vehicles	Ugdb23 with 12 vehicles	0.82
41	Ugdb11dm2 with 5 vehicles	Ugdb11dm1 with 5 vehicles	0.83
42	Uval8A with 3 vehicles	Uval8Adm1 with 3 vehicles	0.89
43	Uval5Adm1 with 3 vehicles	Uval5Adm2 with 3 vehicles	0.92

TABLE II
GP SETTINGS

GP Terminal	Description
CFH	Cost From Here
CFR1	Cost From the closest alternative Route
CR	Cost to Refill
CTD	Cost To Depot
CTT1	Cost To the closest Task
DEM	DEMand
DEM1	DEMand of the closest unserved task
FRT	Fraction of Remaining Tasks
FUT	Fraction of Unassigned Tasks
FULL	FULLness (vehicle load over capacity)
RQ	Remaining Capacity
RQ1	Remaining Capacity of closest alternative route
SC	Serving Cost
ERC	Ephemeral Random Constant number
DC	Deadheading Cost

GP Parameter	Value	GP Function	Description
Population	1024	$+$	Addition
Crossover rate	0.8	$-$	Subtraction
Mutation rate	0.15	$*$	Multiplication
Reproduction rate	0.05	$/$	Protected Division
Number of generations	50	min	Minimum
Elitism	10	max	Maximum
Max depth	8		

and randomness into population, we can create a totally random individual as the mutation result, with the hope that the random individual has a better chance of discovering a less explored region of the search space. The decision of returning a previously seen individual or randomly-created one is made stochastically with a probability of 0.5 (lines 13–18).

IV. EXPERIMENTAL STUDIES

A. Parameter Settings

In this section, we test the effectiveness of the proposed *GPKGS* on a wide range of UCARP transfer optimization scenarios based on the Ugdb, Uval, and Uegl datasets described in [8]. The source and target problems of each scenario are presented in Table I. For each scenario, the "Source/Target Problem" column presents the instance name (its graph topology) and the number of vehicles of the source/target problem to be solved with GPHH. In the table, *dm1* (*dm2*) at the end of a dataset name, e.g., Ugdb11dm2, indicates that the dataset is created by increasing the mean of the random demands in the original dataset by 1 (2). the source and target problems differ in the graph topology, different number of vehicles and even the probability distribution of the demand variable. To calculate the degree of relatedness between the source and target problems, we created 1000 random unique routing policies and evaluated them on the source and target problems. Then, Kendall's rank correlation coefficient [78] between the calculated fitness values of the policies are computed and shown in column *r*. We see a varying degree of correlation but in all cases, the source and target problems are quite related.

For each scenario, we first solve the source problem using GP with population size of 1024 and 50 generations, and then transfer the $1024 \times 50 = 51\,200$ individuals to the target problem. Table II presents the GP parameter settings in our experiments. The division operator / is protected and will return 1 if its denominator is 0. The min (max) function returns the smaller (larger) value of its two input arguments. For training the routing policies, five sampled instances are considered for each generation. The routing policies were tested with 500 samples. These settings are commonly used by previous studies, e.g., [10], [13], [14], [20].

GPKGS requires a set of decision situations Ω to characterize routing policies (Algorithm 4). The accuracy of the dissimilarity measure in (2) depends on the number of decision-making situations utilized for computing it. The accuracy will increase by using more situations but the computational cost will also increase. To construct Ω for each scenario and before starting the experiment, a vehicle was equipped with the path-scanning policy to serve a few tasks and the encountered situations were recorded. For our experiments, we considered 20 situations for characterization of the routing policies, which provided a good balance between accuracy and computational cost.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

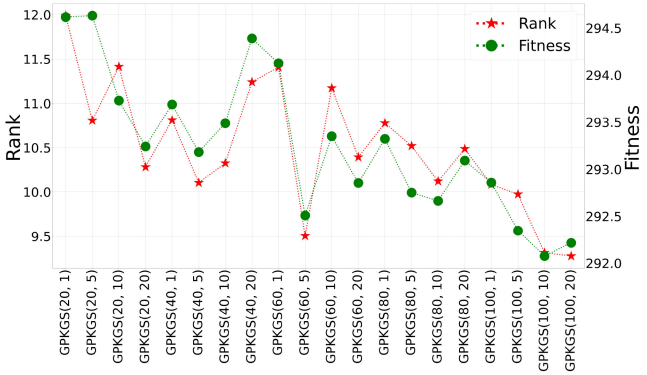


Fig. 3. Performance ranking of *GPKGS*(κ, η) for values of κ and η.

We set the LSH bucket size to $w = 10$ according to the analyses in [69]. The number of hash functions λ and hash tables τ control the accuracy of the compound hashing mechanism. Small values of λ and τ can increase the chance of false collisions and, hence, increase the time required for performing the exact comparisons. However, after a certain point, increasing these values do not have a noticeable impact on the accuracy. In our preliminary testing, we realized that $\lambda = 10$ and $\tau = 100$ yielded good precisions so we used these values for all our experiments.

B. Parameter Sensitivity Analysis

In addition to the conventional GP parameters, *GPKGS* has two extra parameters: 1) the initialization rate κ and 2) the number of trials η, for finding new individuals in `GuidedCrossover` and `GuidedMutation`. To investigate the effect of these parameters on the performance of *GPKGS*, we conducted some pilot experiments on scenarios 1–4 of Table I with $\kappa = \{20, 40, 60, 80, 100\}$ and $\eta = \{1, 5, 10, 20\}$ values. The *GPKGS* is labeled as *GPKGS*(κ, η) for different values of κ and η.

We conducted the Friedman test on the average performance obtained by the tested κ and η values. Fig. 3 presents the fitness and rank of each algorithm setting, averaged over all scenarios. For the majority of the cases, the test reported no significant difference among different κ and η values but we observed that the performance improved as κ increased (κ is percentage and its maximum value is 100). Looking at Fig. 3, we observe that the best average fitness is obtained by *GPKGS*(100, 10) and *GPKGS*(100, 20) while *GPKGS*(100, 20) has a slightly higher (worse) rank. Therefore, we select the $\kappa = 100$ and $\eta = 10$ values for the rest of the article and refer to it simply as *GPKGS* unless otherwise mentioned.

C. Results and Discussions

To verify the effectiveness of *GPKGS*, we compare it with state-of-the-art GP with knowledge transfer methods, i.e., *FullTree* [17], *SubTree* [17], *GATL* [16], and *TLGPC* [38], as well as the GP without knowledge transfer. A summary of these methods is given in Table III.

Table IV presents the fitness of solutions obtained with the compared methods over 30 independent runs. In this table, the minimum mean fitness value is marked in boldface. To investigate if there is a significant difference, the results in Table IV are compared with Friedman's test with a significance level of $\alpha = 0.05$. The ranks of the algorithms and the p-value are given at the bottom of the table. The test's p-value reveals that the difference between the compared algorithms is significant. Additionally, the rank of *GPKGS* is the best. To perform a pairwise comparison, Nemenyi's *post-hoc* test [79] is applied

TABLE III
SUMMARY OF COMPARED EXISTING ALGORITHMS

Algorithm	Subtree selection and transfer mechanism
DDGP [54]	Select the best individuals of the final population into a pool. Choose randomly from the pool to initialise the target GP. Start with a high mutation rate and decrease it gradually
FullTree [17]	Select the best individuals of the final population into a pool. Choose randomly from the pool to initialise the target GP population.
GATL [16]	Select the best and median trees of each generation into a pool. Choose randomly from the pool to initialise the target GP population.
SubTree [17]	Select a random subtree of the final individuals into a pool. Choose randomly from the pool to initialise the target GP population.
TLGPC [38]	Select random subtrees of better-than-average final individuals into a pool. During initialisation and mutation, create a root or subtree randomly or select randomly from the pool.

TABLE IV
TEST PERFORMANCE OF 30 INDEPENDENT RUNS OF THE COMPARED ALGORITHMS (MEAN ± STD)

Scn.	GPHH	GATL [16]	FullTree [17]	SubTree [17]	TLGPC [38]	DDGP [54]	GPKGS
1	60.0±0.1	59.9±0.2	60.0±0.1	60.0±0.2	59.9±0.2	59.9±0.2	**59.8±0.2**
2	91.2±0.1	91.2±0.1	91.2±0.1	91.2±0.1	91.2±0.1	91.2±0.1	**91.1±0.1**
3	58.2±0.1	58.3±0.1	58.3±0.1	58.3±0.1	58.2±0.1	58.2±0.1	**58.1±0.1**
4	432.6±5.5	432.2±5.2	431.1±6.2	433.5±3.7	431.8±6.3	431.4±5.5	**426.9±5.0**
5	576.2±3.9	576.7±3.8	577.6±3.7	577.1±4.9	578.6±4.6	576.4±3.8	**574.1±4.6**
6	125.6±1.2	125.7±0.8	125.6±0.8	125.7±1.0	125.6±1.2	**125.4±0.9**	125.4±0.7
7	249.2±1.7	249.5±1.4	250.1±1.8	250.0±1.9	250.1±1.6	249.4±1.5	**248.9±1.2**
8	444.4±4.7	444.5±5.6	448.0±7.2	446.3±7.1	446.6±6.4	445.4±6.4	**442.0±7.3**
9	324.3±6.2	322.4±5.7	323.2±7.3	325.2±5.5	324.0±6.8	322.6±4.8	**318.0±4.2**
10	358.3±2.6	358.2±3.4	361.2±7.1	**357.0±2.8**	359.1±5.0	360.8±6.6	358.0±5.0
11	340.8±4.4	340.9±4.6	339.5±4.1	341.4±2.9	340.1±5.4	339.9±4.8	**336.1±5.5**
12	351.9±3.5	353.1±4.8	353.5±3.8	353.9±3.9	353.1±3.6	352.1±4.1	**350.3±3.9**
13	356.6±1.6	356.7±1.7	356.9±1.8	357.0±1.8	357.4±1.9	**356.0±1.1**	356.3±1.4
14	310.9±0.5	311.0±0.5	310.2±1.2	311.0±0.7	310.6±1.4	310.0±1.3	**308.2±2.1**
15	342.1±6.2	341.9±5.1	341.9±5.4	344.6±10.6	343.5±7.3	341.8±6.7	**338.7±4.9**
16	360.3±3.1	359.4±4.5	359.9±4.7	360.2±3.2	360.2±3.6	358.4±4.4	**355.2±5.3**
17	359.0±1.8	359.0±1.3	359.6±1.3	359.6±1.5	359.1±1.7	358.8±2.1	**357.3±1.5**
18	363.1±2.8	363.2±4.1	364.0±3.2	362.9±3.1	362.9±4.1	361.3±4.4	**360.9±3.0**
19	382.0±5.5	**380.2±6.0**	384.6±7.3	382.8±6.0	384.5±6.5	382.7±7.7	383.8±6.5
20	444.4±4.7	445.2±6.8	444.2±8.5	445.7±7.5	445.9±7.4	443.5±6.1	**440.8±5.9**
21	382.8±3.3	382.6±2.2	381.6±4.3	385.7±5.4	383.4±4.7	383.0±5.2	**381.6±2.9**
22	**448.2±0.5**	448.4±0.8	450.4±6.6	448.3±0.7	448.6±1.0	449.0±2.3	448.6±1.4
23	166.2±2.0	166.0±1.4	166.2±2.0	165.8±1.4	165.8±2.3	165.4±1.6	**164.8±1.4**
24	321.4±5.2	323.7±5.5	324.1±5.4	323.6±5.0	324.2±4.8	321.8±4.4	**320.9±2.5**
25	369.3±1.8	369.2±2.8	371.1±4.4	369.0±2.5	369.0±2.5	370.0±3.4	**368.0±2.1**
26	389.2±0.2	389.1±0.2	389.2±0.2	389.2±0.2	389.1±0.2	389.1±0.2	**389.1±0.2**
27	376.1±7.6	379.8±7.8	382.0±6.4	378.9±8.3	380.8±6.6	376.3±7.1	**369.9±7.3**
28	384.6±4.4	385.8±6.5	387.6±6.0	388.1±5.2	386.1±5.5	387.2±5.2	**383.1±4.8**
29	351.5±2.5	351.4±1.1	353.0±5.3	351.8±2.1	**350.8±2.3**	351.2±2.0	351.1±1.9
30	347.2±6.1	347.1±6.7	348.1±6.8	348.5±5.7	346.2±6.3	345.8±5.8	**345.6±6.4**
31	326.0±4.7	323.5±4.4	325.0±5.1	326.4±4.6	324.1±5.5	323.4±6.0	**322.6±5.2**
32	165.9±1.8	165.7±1.5	165.4±1.6	165.8±1.8	165.4±1.5	165.0±1.5	**164.7±1.7**
33	415.7±9.2	416.6±8.4	415.4±7.2	418.7±8.1	415.2±7.5	412.8±7.6	**410.7±9.7**
34	360.3±3.1	357.2±4.7	358.3±4.6	358.6±6.9	357.9±5.3	356.4±4.6	**353.6±3.8**
35	383.5±4.2	383.3±2.6	385.0±6.2	383.1±4.0	383.2±3.3	**381.3±2.2**	381.9±2.8
36	351.5±2.5	351.7±2.3	353.0±5.3	**351.1±2.4**	352.0±1.7	352.1±3.1	351.1±1.2
37	196.4±2.4	195.3±2.9	195.2±2.5	194.6±3.1	194.4±2.0	195.8±2.6	**194.0±1.9**
38	382.8±3.3	382.6±4.1	381.6±4.3	383.1±1.7	383.7±4.7	381.2±3.8	**380.0±4.2**
39	249.4±1.2	249.6±1.3	249.7±1.3	249.5±1.5	249.2±1.4	249.2±1.4	**248.4±1.4**
40	248.2±1.8	247.2±1.6	247.1±1.7	247.9±1.6	247.5±1.7	247.2±1.8	**246.3±1.4**
41	462.6±6.0	460.8±5.3	460.3±6.1	460.6±6.2	460.3±6.6	459.6±5.7	**454.8±7.6**
42	426.6±3.3	427.1±2.5	427.9±1.8	427.4±2.6	427.3±1.6	427.8±2.3	**425.8±3.6**
43	499.0±3.9	498.8±4.5	499.6±4.6	499.1±4.9	499.4±4.3	500.0±5.4	**497.2±4.5**
Rank	4.31	4.2	5.26	5.21	4.3	4.58	**1.31**
Friedman's p-value							**1.1e-16**

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

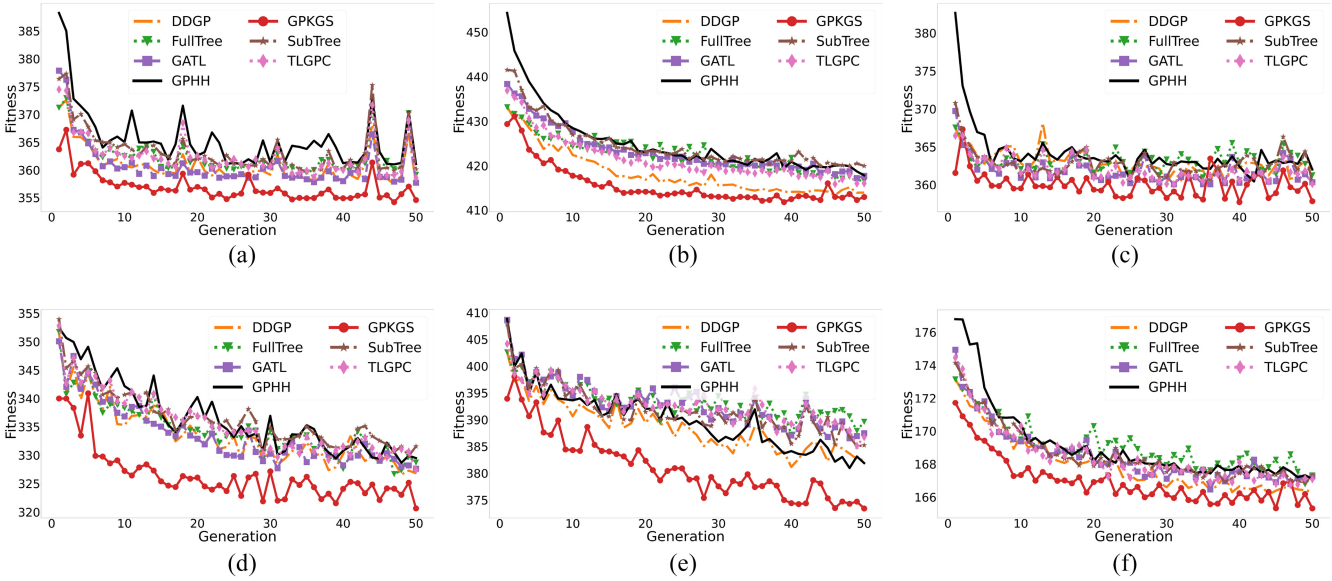


Fig. 4. Convergence curve of *GPKGS* and some existing transfer methods. (a) *Ugdb1*, from 5 to 6 vehicles (Scn. 3). (b) *Ugdb2*, from 6 to 8 vehicles (Scn. 9). (c) *Ugdb3*, from 5 to 7 vehicles (Scn. 13). (d) *Ugdb4*, from 4 to 2 vehicles (Scn. 14). (e) *Ugdb7*, from 5 to 4 vehicles (Scn. 27). (f) *Ugdb21*, from 6 to 4 vehicles (Scn. 34).

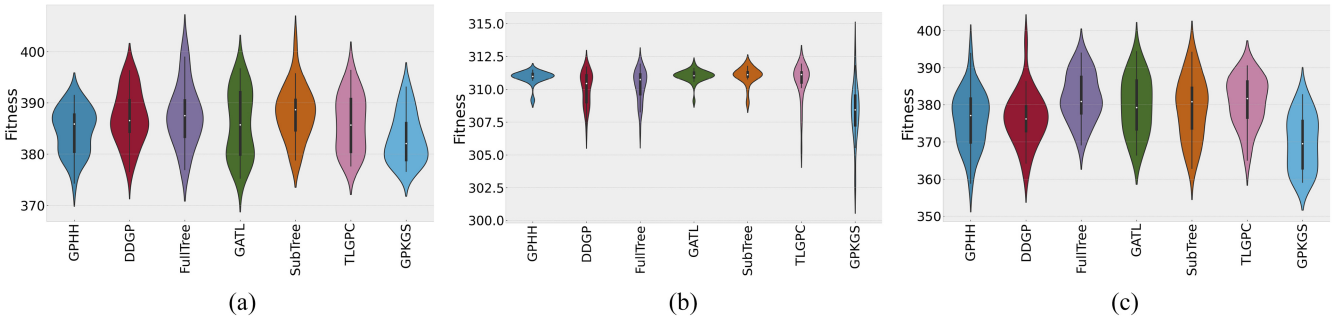


Fig. 5. Performance violin plots of *GPKGS* and the compared transfer algorithms. (a) *Ugdb2*, from 6 to 4 vehicles (Scn. 6). (b) *Ugdb3*, from 5 to 3 vehicles (Scn. 11). (c) *Ugdb7*, from 5 to 4 vehicles (Scn. 27).

TABLE V
POST-HOC COMPARISON OF THE COMPARED EXISTING ALGORITHMS WITH ADJUSTED p-VALUES

	GATL	FullTree	SubTree	TLGPC	DDGP	GPKGS
GPHH	1	0.90	0.96	1	0.18	**0**
GATL	–	0.49	0.52	1	0.37	**0**
FullTree	–	–	1	1	**7e-05**	**0**
SubTree	–	–	–	1	**8e-05**	**0**
TLGPC	–	–	–	–	**0.03**	**0**
DDGP	–	–	–	–	–	**0**

on the results. Table V presents the p-values for the *post-hoc* tests with a significance level of $\alpha = 0.05$ and the significant values are highlighted in boldface. As is evident from the table, the test rejects the null hypotheses for all the comparisons between *GPKGS* and other algorithms and signifies its superiority.

Fig. 4 presents the convergence curve of algorithms for a few scenarios. According to these curves, we see that *GPKGS* starts with a better initial state compared to other existing methods, as well as GPHH with no knowledge transfer. Both *FullTree* and *GPKGS* initialize GP with the best individuals from the source problem. However, *GPKGS* removes phenotypically similar trees and Fig. 4 indicates that the removal of duplicates leads to a better initial solution. This is due to the fact that the duplicates occupy the population without contributing any extra phenotypic information. Hence, when they are removed, other individuals, that may be less fit for the source problem but can perform better for the target problem, will have a chance to be included. Additionally, we see that *GPKGS* manages to retain its initial good performance throughout the evolutionary process. This is in contrast to other methods that may perform better than GPHH initially but then, they gradually lose their superiority and become very similar to GPHH. Although the convergence curve of other scenarios are not given due to space limit, we observed similar patterns in almost all other scenarios too.

Fig. 5 presents the distributions of the performances obtained from 30 independent runs for a few of the experimented scenarios. The plots in this figure also confirm the difference between the *GPKGS* and the compared algorithms. Similar patterns were observed in almost all other scenarios.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

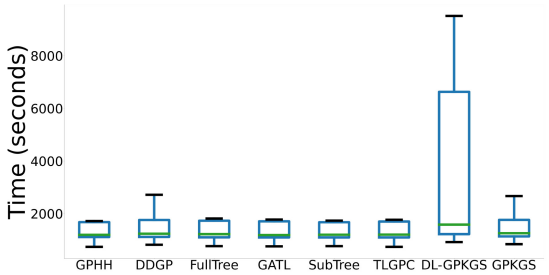


Fig. 6. Training time of the compared algorithms.

D. Training Time

In the guided initialization and guided genetic operators, *GPKGS* needs to search the archive of transferred individuals, $\mathcal{S}$, during crossover and mutation. In our experiments, GP is run for 50 generations for the source problem to evolve populations of 1024 individuals. As a result, $|\mathcal{S}| = 50 \times 1024 = 51\,200$ individuals are transferred for the target problem. Needless to say, linear search of this large set of individuals every time a new individual is created can be daunting. In *GPKGS*, we alleviated this challenge with the LSH technique to hash $\mathcal{S}$ into a hash table $\mathbf{\Psi}$ and search the $\mathbf{\Psi}$ instead. To confirm the challenge and to verify the need for using the hashing method, we disabled the LSH-based search mechanism of Algorithm 6 in *GPKGS* and replaced it with a simple linear search of $\mathcal{S}$. This increased the runtime of *GPKGS* significantly and made the it practically impossible to use. To investigate further, we also considered a version of *GPKGS* that removes any duplicates from $\mathcal{S}$ and performs the linear search on it, instead of the LSH-based search. We refer to this algorithm as *GPKGS Duplicate-free Linear Search* or *DL-GPKGS*. This modification improved the execution time significantly but the execution time was still worse than *GPKGS*. This effect is clearly visible in Fig. 6, which presents the mean training time of each investigated algorithm, averaged over all scenarios.

E. Further Analysis

1) Effectiveness of Guided Search: As described in Section III, *GPKGS* is comprised of three new components: 1) initialization; 2) crossover; and 3) mutation. In this section, we consider the effect of each components on the performance of *GPKGS*. Specifically, we disable the `GuidedCrossover` by considering the ordinary crossover operator and name it *GPKGS-OX*. Similarly, *GPKGS-OM* is the *GPKGS* algorithm with its `GuidedMutation` replaced with the ordinary mutation operator. Additionally, if both the guided crossover and mutation operators are replaced with ordinary operators in *GPKGS*, the knowledge transfer is conducted only by the initialization (Algorithm 2). This method, referred to as *GPKGS-OXM*, can be considered a modified version of the *FullTree* method that does not transfer duplicate individuals and also does not transfer any knowledge about the search space of the source problem. Furthermore, we replaced the guided initialization of *GPKGS* with the *FullTree* method but kept `GuidedCrossover` and `GuidedMutation`. This algorithm is referred to as *GPKGS-FT*. Finally, *GPKGS*-0 stands for the *GPKGS* with $\kappa = 0$, i.e., no initial individual is transferred.

TABLE VI
AVERAGE FITNESS OF 30 INDEPENDENT RUNS OF *GPKGS* AND ITS VARIANTS

Scn.	GPKGS-0	GPKGS-FT	GPKGS-OM	GPKGS-OX	GPKGS-OXM	GPKGS
1	60.0±0.1	59.9±0.2	59.9±0.2	59.9±0.2	59.8±0.2	**59.8±0.2**
2	91.1±0.1	91.1±0.1	91.2±0.1	91.2±0.1	91.2±0.1	**91.1±0.1**
3	58.2±0.1	58.2±0.1	58.2±0.1	58.2±0.1	58.2±0.2	**58.1±0.1**
4	**424.5±5.2**	427.7±6.0	425.9±6.4	427.5±4.9	428.8±6.9	426.9±5.0
5	576.0±4.0	574.9±6.2	575.1±3.8	576.3±4.6	575.4±5.2	**574.1±4.6**
6	125.5±1.0	125.3±0.7	**125.2±0.6**	125.7±0.9	125.4±0.9	125.4±0.7
7	249.4±1.7	250.0±1.7	249.5±1.8	249.1±1.6	249.4±1.9	**248.9±1.2**
8	443.5±4.7	446.8±7.8	446.8±7.8	444.4±8.1	447.4±9.2	**442.0±7.3**
9	322.5±6.5	321.6±5.5	319.9±5.5	319.3±5.3	320.8±5.9	**318.0±4.2**
10	**355.9±2.7**	358.0±5.6	356.4±4.5	356.9±3.4	358.4±5.4	358.0±5.0
11	340.8±6.5	338.2±5.1	337.4±6.2	337.2±6.1	338.7±3.8	**336.1±5.5**
12	350.7±3.0	351.7±4.0	351.0±2.5	**350.2±3.0**	350.4±1.6	350.3±3.9
13	356.2±1.2	356.6±1.7	356.5±1.6	**356.1±1.3**	356.4±1.6	356.3±1.4
14	310.6±2.5	309.9±1.8	308.6±1.8	308.9±2.4	308.9±1.8	**308.2±2.1**
15	340.1±5.9	340.5±4.9	**337.7±4.5**	338.6±7.1	338.8±6.5	338.7±4.9
16	356.7±3.8	357.0±4.8	**355.0±5.2**	355.6±4.0	357.0±4.4	355.2±5.3
17	358.5±1.9	358.4±1.8	357.5±2.1	357.7±1.6	358.1±1.3	**357.3±1.5**
18	361.5±4.5	362.2±3.3	361.8±1.8	361.0±3.7	361.5±2.1	**360.9±3.0**
19	384.4±8.0	386.2±6.7	384.4±6.3	384.6±6.0	383.4±6.1	383.8±6.5
20	441.8±5.0	442.4±7.4	441.7±5.4	441.3±6.4	441.2±7.0	**440.8±5.9**
21	382.4±4.9	383.8±4.1	381.9±3.9	**381.2±3.4**	381.8±3.9	381.6±2.9
22	449.0±1.9	449.7±2.6	449.0±1.6	448.4±1.3	448.4±0.7	448.6±1.4
23	165.2±1.5	165.4±1.8	165.1±1.8	**164.7±2.0**	165.0±1.5	164.8±1.4
24	321.3±4.1	323.9±5.1	322.2±2.9	321.7±3.5	321.0±2.9	**320.9±2.5**
25	368.8±2.2	370.9±4.1	368.5±2.4	368.9±2.3	368.2±2.2	**368.0±2.1**
26	389.1±0.1	389.2±0.2	389.1±0.2	389.1±0.1	389.1±0.2	**389.1±0.2**
27	370.0±7.2	377.7±7.7	373.1±7.1	373.1±7.8	376.2±7.6	**369.9±7.3**
28	383.4±6.1	387.7±5.7	387.1±5.0	383.6±4.5	384.6±5.0	**383.1±4.8**
29	351.7±1.0	350.7±2.2	351.2±1.1	**350.7±2.7**	351.1±1.9	351.1±1.9
30	345.6±6.4	**345.2±6.2**	346.6±4.6	345.4±6.0	347.0±5.9	345.6±6.4
31	324.6±4.5	325.1±3.5	324.2±4.5	**322.1±4.8**	322.7±4.8	322.6±5.2
32	165.2±1.5	165.2±1.2	165.1±1.6	165.3±1.0	164.9±1.4	**164.7±1.7**
33	411.0±10.4	**410.7±8.2**	412.6±7.3	411.7±6.5	411.9±6.6	410.7±9.7
34	354.9±4.5	356.3±5.4	354.1±3.9	354.4±3.9	353.8±4.0	**353.6±3.8**
35	381.8±2.7	383.1±2.7	381.9±2.4	**381.6±2.3**	382.0±2.0	381.9±2.8
36	351.6±1.2	352.0±3.2	**351.1±1.2**	351.3±2.1	351.1±2.2	351.1±1.2
37	194.9±2.9	195.3±2.4	195.3±2.4	195.2±2.3	194.9±2.2	**194.0±1.9**
38	381.9±4.0	381.1±3.4	381.0±3.2	380.4±3.2	380.1±4.3	**380.0±4.2**
39	249.2±1.4	249.4±1.3	248.5±1.2	248.5±1.1	**248.3±1.4**	248.4±1.4
40	247.6±2.2	247.0±2.1	246.6±1.5	246.7±1.1	246.8±1.4	**246.3±1.4**
41	457.9±8.0	459.5±5.8	457.4±7.2	458.2±7.2	458.6±6.3	**454.8±7.6**
42	426.0±4.6	426.7±4.0	425.8±4.3	**425.2±3.9**	426.5±3.1	425.8±3.6
43	500.5±4.6	498.1±4.8	498.4±4.6	496.8±3.3	**496.6±3.8**	497.2±4.5
Rank	4.15	5.33	3.71	3.13	3.8	**1.87**
Friedman's p-value						**1.1102e-16**

Table VI presents the average performance of the algorithms in which the best fitness values are highlighted in boldface. To perform statistical analysis, we included the performance of GPHH from Table IV (not shown again in Table VI to save space) and conducted the Friedman test. GPHH was ranked 6.01 in this context. The ranks of other methods, as well as the p-value, are given in Table VI. As can be seen from Table VI, *GPKGS* has the best rank amongst the algorithms. Since the p-value indicates the existence of significant difference, the Nemenyi *post-hoc* comparison is performed and the obtained p-values are given in Table VII.

By comparing *GPKGS* and *GPKGS*-0, we observe that when the transfer of genetic materials is disabled, the effectiveness of the resulting algorithm, *GPKGS*-0, degrades significantly. This observation indicates the importance of genetic materials. However, relying solely on knowledge-guided initialization, crossover, and mutation, and without any transfer of genetic

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

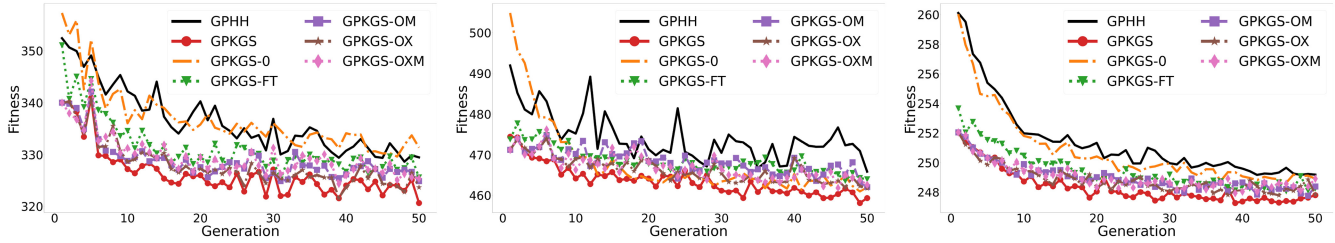


Fig. 7. Convergence curve of *GPKGS* and its variants. (a) *Ugdb4*, from 4 to 2 vehicles (Scn. 14). (b) From *Ugdb11dm2* with 5 vehicles to *Ugdb11dm1* 5 vehicles (Scn. 29). (c) *Ugdb23*, from 10 to 12 vehicles (Scn. 37).

TABLE VII
p-VALUES FOR THE POST-HOC COMPARISON OF THE *GPKGS* AND ITS VARIANTS

	GPHH	GPKGS-0	GPKGS-FT	GPKGS-OM	GPKGS-OX	GPKGS-OXM	GPKGS
GPHH	–	**13e-5**	1	**2e-05**	**0**	**4e-05**	**0**
GPKGS-0	–	–	0.26	1	0.59	1	**2e-05**
GPKGS-FT	–	–	–	**0.01**	**5e-05**	**0.02**	**0**
GPKGS-OM	–	–	–	–	1	1	**16e-5**
GPKGS-OX	–	–	–	–	–	1	0.15
GPKGS-OXM	–	–	–	–	–	–	**72e-5**

TABLE VIII
PERFORMANCE STATISTICS OF `GuidedCrossover` AND `GuidedMutation`

`GuidedCrossover`					
	0 Unseen Offspring	1 Unseen Offspring	2 Unseen Offspring	Total Seen	Total Unseen
Mean	18.00	36.06	22037.94	15937.69	44111.96
Std	24.90	28.18	64.19	6060.67	107.39

`GuidedMutation`			
	Total Random	Total Seen	Total Unseen
Mean	5.58	1510.63	3331.22
Std	14.85	975.11	1348.89

materials, *GPKGS*-0 still manages to outperform GPHH, which highlights the effectiveness of guided initialization and search even if no genetic materials are transferred.

Furthermore, *GPKGS-FT* performs statistically similar to GPHH but significantly worse than *GPKGS*. *GPKGS-FT* is a version of *GPKGS* with guided crossover and mutation but without duplicate removal during initialization. The fact that it performs worse than *GPKGS* but similar to GPHH indicates how detrimental the presence of duplicates can be to the effectiveness of knowledge transfer. This is further confirmed when we note that *GPKGS*-0, which does not transfer any genetic materials, outperforms GPHH. The presence of duplicates contributes to the lack of diversity in initial population and gives GP a negative bias toward the smaller regions of the search space that the duplicates represent. The negative effect of duplicates in the initial population can be understood even better when we note that *GPKGS-OXM*, which transfers only the unique individuals without any guided search, has a significantly better performance than *GPKGS-FT*.

By comparing *GPKGS-OXM* and GPHH, we can further verify the effectiveness of duplicate removal. However, *GPKGS-OXM* performs significantly worse than *GPKGS* which indicates that, despite the effectiveness of the duplicate removal, the search performance can be improved significantly when it is guided with knowledge. In this regard and by comparing the performance of *GPKGS-OM*, *GPKGS-OX*, and *GPKGS-OXM*, we note that the guided mutation is more effective than the guided crossover because *GPKGS-OX*, which utilizes guided mutation but an ordinary crossover, performs significantly better than *GPKGS-OXM* and similarly to *GPKGS*. *GPKGS-OM*, on the other hand, cannot reach the performance of *GPKGS*, although it outperforms *GPKGS-OXM*. This result can be explained when the role of the mutation operator is considered as the main way that GP can introduce diversity into its population to escape from local optima. This role becomes more prominent when we know that GP may lose its population diversity and can converge to local optima when it solves UCARP [13], [14]. Fig. 7 presents the convergence curves of *GPKGS* and its variants.

2) Computational Advantage: The `GuidedCrossover` and `GuidedMutation` operators attempt η times at finding unseen routing policies and discard the seen ones until either enough unseen individuals are created or the threshold is reached and seen results are accepted. The goal here is to avoid wasting computational resources on evaluating the individuals that are likely to have bad quality and use the saved resources better on evaluating new routing policies that are not seen previously. Table VIII presents the average number of times that `GuidedCrossover` and `GuidedMutation` found seen and unseen individuals over all our experiments. As can be seen, during a *GPKGS* run, `GuidedCrossover` encounters 44 131.89 unseen and 15 064.29 seen individuals on average. As a result, fitness evaluation is avoided for 15 064.29 previously seen policies. Additionally, there were $11.84 + 29.09$ cases that `GuidedCrossover` could not find unseen offspring and previously seen individuals were accepted instead.

Similarly, `GuidedMutation` discards most of the 1649.22 previously seen individuals that it runs into and manages to find 4138.96 new policies on average. Additionally, there were 2.67 average number of times that `GuidedMutation` could not find any new individual and had to create a totally random policy. Hence, for the majority of cases, `GuidedMutation` managed to find unseen individuals and there were no need for creating random policies.

3) Duplicates in Knowledge Source: The motivation for proposing the duplicate removal mechanism in Section III-B1

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

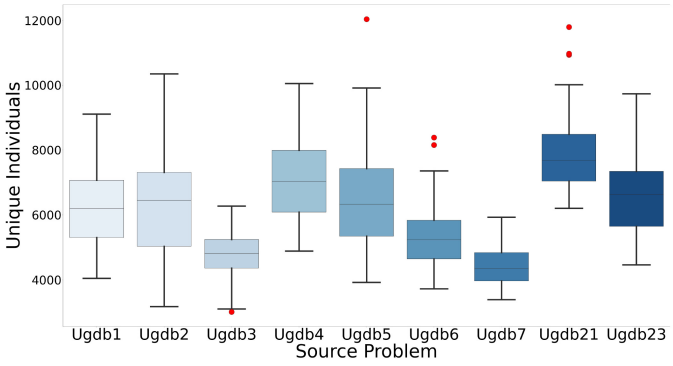


Fig. 8. Number of available unique individuals in the knowledge source.

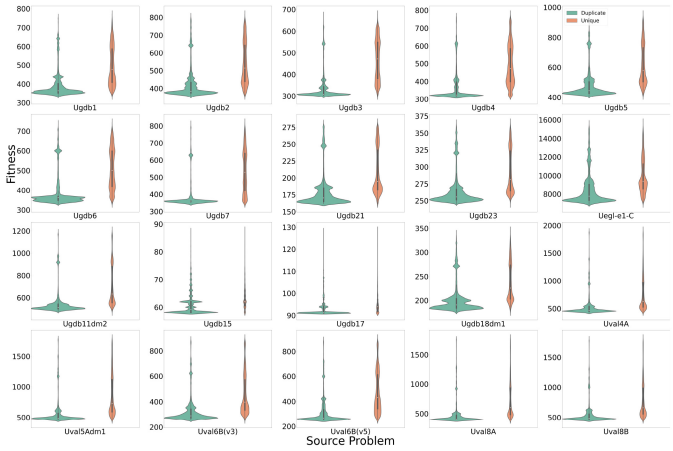


Fig. 9. Distribution of unique and duplicate individuals in source problem.

is that a GP knowledge source can be inflicted with redundancy. To investigate this in our experiments, we recorded the number of unique individuals that were found in each source problem as defined in Table I, averaged them over all runs and summarized the results into the box plots in Fig. 8. Considering that there are $1024 \times 50 = 51\,200$ transferred individuals according to our experiment settings, it is obvious from the plots that in almost all scenarios, the source contained a large number of duplicates to the point that at best, 24% of the individuals were unique (*Ugdb5*). For some cases like *Ugdb3*, this rate fell even below 10%. To investigate further, we plotted the distribution of the unique and duplicate individuals in each source problem against their fitness value in Fig. 9. As is evident, the majority of high-quality individuals were duplicates, transfer of which will amount to transfer of redundant knowledge that cannot be helpful for solving the target problem. As a result, the performance of any method, such as FullTree, that does not consider this redundancy, will suffer. This can be deducted from the results in Tables IV–VII. While the performance of FullTree is statistically similar to the performance of GPHH, *GPKGS-OXM* that transfers unique individuals without any guided search, outperforms GPHH.

V. Conclusion

In this article, we aimed to propose a novel approach to transfer optimization for GP to evolve effective routing policies for UCARP. We achieved this goal by devising an algorithm that performs knowledge transfer through two components: 1) transfer of unique genetic materials of a diverse set of good solutions and 2) transfer of an approximate overview of the areas of the search space that has low potential to be explored for the target problem, since its useful information has already been transferred in the initialization. We developed an elaborated hashing method that enabled GP to efficiently compare solutions and avoid creating new individuals that are likely to be in a previously-explored area. Our experimental studies showed that the proposed algorithm improves the effectiveness of GP significantly on almost all the test datasets and outperforms the state-of-the-art transfer optimization methods for GP. Furthermore, through a wide range of control experiments, we verified the effectiveness of each proposed novel component of the algorithm.

This work has the following two key findings. First, the detection of duplicates in the source knowledge plays a crucial role in the effectiveness of knowledge transfer and significant improvement can be achieved in the quality of knowledge transfer if the duplicates are removed. Second, the transferred knowledge could be used in different ways rather than only initializing individuals. Given that the promising source individuals have been transferred to generate the initial target individuals, the other source individuals, regardless of their quality, can be used to encourage the search to explore new regions and solve the target problem better.

For future work, one direction is to gain deeper understanding about the source problem, e.g., through fitness landscape analysis, and transfer the higher level knowledge. Additionally, we will consider more guided approaches to directly generate unseen offspring in the source problem, rather than randomly generating and doing revisit check. One possible way is to use semantics [27].

Another future direction is to investigate the effectiveness of fitness as a transfer criterion. Fitness may not be the only indicator of transferability, and there may be other criteria (e.g., diversity) for the potential of reaching better final solution. We will investigate this and design better transfer criteria.

A third future direction will be to consider multiple source problems instead of a single one. We will explore how to tackle the challenge if there are many source problems, but only a few of them are related to the target problem. We will consider offline and online similarity measuring approaches and adapt the GP search to the problems with multiple sources based on the different similarities of the source problems.

References

[1] B. L. Golden and R. T. Wong, "Capacitated arc routing problems," *Networks*, vol. 11, no. 3, pp. 305–315, 1981.
[2] S. Wøhlk, *A Decade of Capacitated Arc Routing*. Boston, MA, USA: Springer, 2008, pp. 29–48.
[3] J. F. Campbell and A. Langevin, *Roadway Snow and Ice Control*. Boston, MA, USA: Springer, 2000, pp. 389–418.
[4] L. Chen, M. Gendreau, M. H. Hà, and A. Langevin, "A robust optimization approach for the road network daily maintenance routing problem with uncertain service time," *Transp. Res. E, Logist. Transp. Rev.*, vol. 85, pp. 40–51, Jan. 2016.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

[5] L. Chen, M. H. Hà, A. Langevin, and M. Gendreau, "Optimizing road network daily maintenance operations with stochastic service and travel times," *Transp. Res. E, Logist. Transp. Rev.*, vol. 64, pp. 88–102, Apr. 2014.

[6] S. K. Amponsah and S. Salhi, "The investigation of a class of capacitated arc routing problems: The collection of garbage in developing countries," *Waste Manage.*, vol. 24, no. 7, pp. 711–721, 2004.

[7] E. B. Tirkolaee, I. Mahdavi, M. M. S. Esfahani, and G. W. Weber, "A hybrid augmented ant colony optimization for the multi-trip capacitated arc routing problem under fuzzy demands for urban solid waste management," *Waste Manage. Res.*, vol. 38, no. 2, pp. 156–172, 2020.

[8] Y. Mei, K. Tang, and X. Yao, "Capacitated arc routing problem in uncertain environments," in *Proc. IEEE Congr. Evol. Comput.*, Barcelona, Spain, 2010, pp. 1–8.

[9] J. Liu, K. Tang, and X. Yao, "Robust optimization in uncertain capacitated arc routing problems: Progresses and perspectives [review article]," *IEEE Comput. Intell. Mag.*, vol. 16, no. 1, pp. 63–82, Feb. 2021.

[10] Y. Liu and Y. Mei, "Automated heuristic design using genetic programming hyper-heuristic for uncertain capacitated arc routing problem," in *Proc. Genet. Evol. Comput. Conf.*, 2017, pp. 290–297.

[11] S. J. Louis and J. McDonnell, "Learning with case-injected genetic algorithms," *IEEE Trans. Evol. Comput.*, vol. 8, no. 4, pp. 316–328, Aug. 2004.

[12] A. Gupta, Y.-S. Ong, and L. Feng, "Insights on transfer optimization: Because experience is the best teacher," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 1, pp. 51–64, Feb. 2018.

[13] M. A. Ardeh, Y. Mei, and M. Zhang, "Transfer learning in genetic programming hyper-heuristic for solving uncertain capacitated arc routing problem," in *Proc. IEEE Congr. Evol. Comput.*, Wellington, New Zealand, 2019, pp. 49–56.

[14] M. A. Ardeh, Y. Mei, and M. Zhang, "Genetic programming hyper-heuristic with knowledge transfer for uncertain capacitated arc routing problem," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2019, pp. 334–335.

[15] M. A. Ardeh, Y. Mei, and M. Zhang, "A novel genetic programming algorithm with knowledge transfer for uncertain capacitated arc routing problem," in *Proc. Pac. Rim Int. Conf. Artif. Intell.*, 2019, pp. 196–200.

[16] B. Koçer and A. Arslan, "Genetic transfer learning," *Expert Syst. Appl.*, vol. 37, no. 10, pp. 6997–7002, 2010.

[17] T. T. H. Dinh, T. H. Chu, and Q. U. Nguyen, "Transfer learning in genetic programming," in *Proc. IEEE Congr. Evol. Comput.*, Sendai, Japan, 2015, pp. 1145–1151.

[18] J. Wang, K. Tang, J. A. Lozano, and X. Yao, "Estimation of the distribution algorithm with a stochastic local search for uncertain capacitated arc routing problems," *IEEE Trans. Evol. Comput.*, vol. 20, no. 1, pp. 96–109, Feb. 2016.

[19] J. Wang, K. Tang, and X. Yao, "A memetic algorithm for uncertain capacitated arc routing problems," in *Proc. IEEE Workshop Memetic Comput.*, Singapore, 2013, pp. 72–79.

[20] Y. Mei and M. Zhang, "Genetic programming hyper-heuristic for multi-vehicle uncertain capacitated arc routing problem," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2018, pp. 141–142.

[21] J. MacLachlan, Y. Mei, J. Branke, and M. Zhang, "Genetic programming hyper-heuristics with vehicle collaboration for uncertain capacitated arc routing problems," *Evol. Comput.*, vol. 28, no. 4, pp. 563–593, 2019, doi: 10.1162/evco_a_00267.

[22] S. Wang, Y. Mei, and M. Zhang, "Novel ensemble genetic programming hyper-heuristics for uncertain capacitated arc routing problem," in *Proc. Genet. Evol. Comput. Conf.*, 2019, pp. 1093–1101.

[23] S. Wang, Y. Mei, and M. Zhang, "Towards interpretable routing policy: A two stage multi-objective genetic programming approach with feature selection for uncertain capacitated arc routing problem," in *Proc. Symp. Ser. Comput. Intell.*, Canberra, ACT, Australia, 2021, pp. 2399–2406.

[24] S. Wang, Y. Mei, M. Zhang, and X. Yao, "Genetic programming with niching for uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, early access, Jul. 7, 2021, doi: 10.1109/TEVC.2021.3095261.

[25] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge MA, USA: MIT Press, 1992.

[26] A. De Lorenzo, A. Bartoli, M. Castelli, E. Medvet, and B. Xue, "Genetic programming in the twenty-first century: A bibliometric and content-based analysis from both sides of the fence," *Genet. Program. Evol. Mach.*, vol. 21, pp. 181–204, Jul. 2019.

[27] Q. Chen, B. Xue, and M. Zhang, "Improving generalization of genetic programming for symbolic regression with angle-driven geometric semantic operators," *IEEE Trans. Evol. Comput.*, vol. 23, no. 3, pp. 488–502, Jun. 2019.

[28] Y. Bi, B. Xue, and M. Zhang, "Genetic programming with a new representation to automatically learn features and evolve ensembles for image classification," *IEEE Trans. Cybern.*, vol. 51, no. 4, pp. 1769–1783, Apr. 2021, doi: 10.1109/tcyb.2020.2964566.

[29] S. Nguyen, M. Zhang, and K. C. Tan, "Surrogate-assisted genetic programming with simplified models for automated design of dispatching rules," *IEEE Trans. Cybern.*, vol. 47, no. 9, pp. 2951–2965, Sep. 2017.

[30] J. Park, Y. Mei, S. Nguyen, G. Chen, and M. Zhang, "An investigation of ensemble combination schemes for genetic programming based hyper-heuristic approaches to dynamic job shop scheduling," *Appl. Soft Comput. J.*, vol. 63, pp. 72–86, Feb. 2018.

[31] E. K. Burke, M. R. Hyde, G. Kendall, G. Ochoa, E. Ozcan, and J. R. Woodward, *Exploring Hyper-Heuristic Methodologies With Genetic Programming*. Heidelberg, Germany: Springer, 2009, pp. 177–201.

[32] P. Ross, "Hyper-heuristics," in *Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques*, 2nd ed. Boston, MA, USA: Springer, 2014, pp. 611–638.

[33] E. K. Burke, M. R. Hyde, G. Kendall, G. Ochoa, E. Özcan, and J. R. Woodward, *A Classification of Hyper-Heuristic Approaches: Revisited* (International Series in Operations Research and Management Science), vol. 272. New York, NY, USA: Springer, 2019, pp. 453–477.

[34] F. Zhang, Y. Mei, S. Nguyen, and M. Zhang, "Evolving scheduling heuristics via genetic programming with feature selection in dynamic flexible job-shop scheduling," *IEEE Trans. Cybern.*, vol. 51, no. 4, pp. 1797–1811, Apr. 2021, doi: 10.1109/tcyb.2020.3024849.

[35] B. Tan, H. Ma, Y. Mei, and M. Zhang, "A cooperative coevolution genetic programming hyper-heuristic approach for on-line resource allocation in container-based clouds," *IEEE Trans. Cloud Comput.*, early access, Sep. 24, 2020, doi: 10.1109/TCC.2020.3026338.

[36] P. Cunningham and B. Smyth, "Case-based reasoning in scheduling: Reusing solution components," *Int. J. Prod. Res.*, vol. 35, no. 11, pp. 2947–2962, 1997.

[37] M. E. Taylor, S. Whiteson, and P. Stone, "Transfer learning for policy search methods," in *Proc. 23rd Int. Conf. Mach. Learn.*, 2006, pp. 1–4.

[38] M. Iqbal, B. Xue, H. Al-Sahaf, and M. Zhang, "Cross-domain reuse of extracted knowledge in genetic programming for image classification," *IEEE Trans. Evol. Comput.*, vol. 21, no. 4, pp. 569–587, Aug. 2017.

[39] M. Iqbal, W. N. Browne, and M. Zhang, "Reusing building blocks of extracted knowledge to solve complex, large-scale boolean problems," *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 465–480, Aug. 2014.

[40] D. O'Neill, H. Al-Sahaf, B. Xue, and M. Zhang, "Common subtrees in related problems: A novel transfer learning approach for genetic programming," in *Proc. IEEE Congr. Evol. Comput.*, Donostia, Spain, 2017, pp. 1287–1294.

[41] E. Haslam, B. Xue, and M. Zhang, "Further investigation on genetic programming with transfer learning for symbolic regression," in *Proc. IEEE Congr. Evol. Comput.*, Vancouver, BC, Canada, 2016, pp. 3598–3605.

[42] B. Da, A. Gupta, and Y.-S. Ong, "Curbing negative influences online for seamless transfer evolutionary optimization," *IEEE Trans. Cybern.*, vol. 49, no. 12, pp. 4365–4378, Dec. 2019.

[43] L. Feng, Y.-S. Ong, A.-H. Tan, and I. W. Tsang, "Memes as building blocks: A case study on evolutionary optimization + transfer learning for routing problems," *Memetic Comput.*, vol. 7, no. 3, pp. 159–180, 2015.

[44] L. Feng, Y.-S. Ong, M.-H. Lim, and I. W. Tsang, "Memetic search with interdomain learning: A realization between CVRP and CARP," *IEEE Trans. Evol. Comput.*, vol. 19, no. 5, pp. 644–658, Oct. 2015.

[45] L. Feng, Y.-S. Ong, S. Jiang, and A. Gupta, "Autoencoding evolutionary search with learning across heterogeneous problems," *IEEE Trans. Evol. Comput.*, vol. 21, no. 5, pp. 760–772, Oct. 2017.

[46] L. Feng *et al.*, "Evolutionary multitasking via explicit autoencoding," *IEEE Trans. Cybern.*, vol. 49, no. 9, pp. 3457–3470, Sep. 2019.

[47] L. Zhou *et al.*, "Solving dynamic vehicle routing problem via evolutionary search with learning capability," in *Proc. IEEE Congr. Evol. Comput.*, Donostia, Spain, 2017, pp. 890–896.

[48] M. Pelikan, M. W. Hauschild, and P. L. Lanzi, *Transfer Learning, Soft Distance-Based Bias, and the Hierarchical BOA* (Lecture Notes in Computer Science, 7491). Heidelberg, Germany: Springer, 2012, pp. 173–183.

[49] M. Pelikan and M. W. Hauschild, "Distance-based bias in model-directed optimization of additively decomposable problems," in *Proc. 14th Int. Conf. Genet. Evol. Comput.*, 2012, pp. 273–280.

[50] M. W. Hauschild and M. Pelikan, "Intelligent bias of network structures in the hierarchical BOA," in *Proc. 11th Annu. Genet. Evol. Comput. Conf.*, 2009, pp. 413–420.

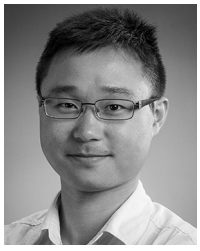
Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.

[51] M. W. Hauschild, K. Sastry, M. Pelikan, and D. E. Goldberg, "Using previous models to bias structural learning in the hierarchical BOA," in *Proc. 10th Annu. Conf. Genet. Evol. Comput.*, 2008, pp. 415–422.
[52] M. A. Ardeh, Y. Mei, and M. Zhang, "Genetic programming hyper-heuristics with probabilistic prototype tree knowledge transfer for uncertain capacitated arc routing problems," in *Proc. IEEE Congr. Evol. Comput.*, Glasgow, U.K., 2020, pp. 1–8.
[53] T. T. Nguyen, S. Yang, and J. Branke, "Evolutionary dynamic optimization: A survey of the state of the art," *Swarm Evol. Comput.*, vol. 6, pp. 1–24, Oct. 2012.
[54] M. A. Ardeh, Y. Mei, and M. Zhangz, "Diversity-driven knowledge transfer for GPHH to solve uncertain capacitated arc routing problem," in *Proc. IEEE Symp. Ser. Comput. Intell.*, Canberra, ACT, Australia, 2020, pp. 2407–2414.
[55] L. Muñoz, L. Trujillo, and S. Silva, "Transfer learning in constructive induction with genetic programming," *Genet. Program. Evol. Mach.*, vol. 21, pp. 529–569, Dec. 2020.
[56] B. Al-Helali, Q. Chen, B. Xue, and M. Zhang, "Multi-tree genetic programming with new operators for transfer learning in symbolic regression with incomplete data," *IEEE Trans. Evol. Comput.*, early access, May 12, 2021, doi: 10.1109/TEVC.2021.3079843.
[57] A. Gupta, Y.-S. Ong, and L. Feng, "Multifactorial evolution: Toward evolutionary multitasking," *IEEE Trans. Evol. Comput.*, vol. 20, no. 3, pp. 343–357, Jun. 2016.
[58] K. K. Bali, Y.-S. Ong, A. Gupta, and P. S. Tan, "Multifactorial evolutionary algorithm with online transfer parameter estimation: MFEA-II," *IEEE Trans. Evol. Comput.*, vol. 24, no. 1, pp. 69–83, Feb. 2020.
[59] L. Feng *et al.*, "Explicit evolutionary multitasking for combinatorial optimization: A case study on capacitated vehicle routing problem," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3143–3156, Jun. 2021, doi: 10.1109/tcyb.2019.2962865.
[60] L. Zhou *et al.*, "Toward adaptive knowledge transfer in multifactorial evolutionary computation," *IEEE Trans. Cybern.*, vol. 51, no. 5, pp. 2563–2576, May 2021, doi: 10.1109/tcyb.2020.2974100.
[61] L. Feng *et al.*, "Towards faster vehicle routing by transferring knowledge from customer representation," *IEEE Trans. Intell. Transp. Syst.*, early access, Sep. 10, 2020, doi: 10.1109/tits.2020.3018903.
[62] L. Feng *et al.*, "Solving generalized vehicle routing problem with occasional drivers via evolutionary multitasking," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3171–3184, Jun. 2021.
[63] T. Wei and J. Zhong, "A preliminary study of knowledge transfer in multi-classification using gene expression programming," *Front. Neurosci.*, vol. 13, p. 1396, Jan. 2020.
[64] R. K. Arakaki and F. L. Usberti, "An efficiency-based path-scanning heuristic for the capacitated arc routing problem," *Comput. Oper. Res.*, vol. 103, pp. 288–295, Mar. 2019.
[65] T. Hildebrandt and J. Branke, "On using surrogates with genetic programming," *Evol. Comput.*, vol. 23, no. 3, pp. 343–367, 2015.
[66] Y. Mei, S. Nguyen, B. Xue, and M. Zhang, "An efficient feature selection algorithm for evolving job shop scheduling rules with genetic programming," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 1, no. 5, pp. 339–353, Oct. 2017.
[67] M. R. Abbasifard, B. Ghahremani, and H. Naderi, "A survey on nearest neighbor search methods," *Int. J. Comput. Appl.*, vol. 95, no. 25, pp. 39–52, 2014.
[68] G. Shakhnarovich, T. Darrell, and P. Indyk, *Nearest-Neighbor Methods in Learning and Vision: Theory and Practice* (Neural Information Processing). Cambridge, MA, USA: MIT Press, 2006.
[69] M. Datar, P. Indyk, N. Immorlica, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions," in *Proc. Annu. Symp. Comput. Geom.*, 2004, pp. 253–262.
[70] P. Indyk and R. Motwd, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. 30th Annu. ACM Symp. Theory Comput. (STOC)*, 1998, pp. 604–613.
[71] R. Weber, H.-J. Schek, and S. Blott, "A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces," in *Proc. 24th Int. Conf. Very Large Data Bases (VLDB)*, 1998, pp. 194–205.
[72] S. Arya, D. M. Mount, N. S. Netanyahu, R. Silverman, and A. Y. Wu, "An optimal algorithm for approximate nearest neighbor searching fixed dimensions," *J. ACM*, vol. 45, no. 6, pp. 891–923, 1998.
[73] E. Kushilevitz, R. Ostrovsky, and Y. Rabani, "Efficient search for approximate nearest neighbor in high dimensional spaces," in *Proc. Conf. Annu. ACM Symp. Theory Comput.*, 1998, pp. 614–623.
[74] S. Har-Peled, "A replacement for Voronoi diagrams of near linear size," in *Proc. IEEE Annu. Symp. Found. Comput. Sci.*, Newport Beach, CA, USA, 2001, pp. 94–103.
[75] J. M. Kleinberg, "Two algorithms for nearest-neighbor search in high dimensions," in *Proc. Annu. ACM Symp. Theory Comput.*, 1997, pp. 599–608.
[76] J. Gan, J. Feng, Q. Fang, and W. Ng, "Locality-sensitive hashing scheme based on dynamic collision counting," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2012, pp. 541–552.
[77] Q. Huang, J. Feng, Q. Fang, W. Ng, and W. Wang, "Query-aware locality-sensitive hashing scheme for l_p norm," *VLDB J.*, vol. 26, no. 5, pp. 683–708, 2017.
[78] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, nos. 1–2, pp. 81–93, 1938.
[79] P. Nemenyi, "Distribution-free multiple comparisons," Ph.D. dissertation, Dept. Mathematics, Princeton Univ., Princeton, NJ, USA, 1963.



Mazhar Ansari Ardeh (Graduate Student Member, IEEE) received the B.Sc. degree from the University of Sistan and Baluchestan, Zahedan, Iran, in 2008, and the M.Sc. degree from Kharazmi University, Tehran, Iran, in 2014. He is currently pursuing the Ph.D. degree in computer science with the School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand.

His current research interests include evolutionary computation, combinatorial optimization, hyper-heuristic learning/optimization, and arc and vehicle routing, transfer, and multitask optimization.



Yi Mei (Senior Member, IEEE) received the B.Sc. and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 2005 and 2010, respectively.

He is a Senior Lecturer with the School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand. He has more than 100 fully referred publications, including the top journals in EC and Operations Research, such as IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, IEEE TRANSACTIONS ON CYBERNETICS, and *ACM Transactions on Mathematical Software*. His research interests include evolutionary scheduling and combinatorial optimization, machine learning, genetic programming, and hyperheuristics.

Dr. Mei was the Vice-Chair of the IEEE CIS Emergent Technologies Technical Committee and a member of the Intelligent Systems Applications Technical Committee. He is an editorial board member/associate editor of three international journals, and a Guest Editor of a special issue of the *Genetic Programming and Evolvable Machines* journal.



Mengjie Zhang (Fellow, IEEE) received the B.E. and M.E. degrees from the Artificial Intelligence Research Center, Agricultural University of Hebei, Baoding, China, in 1989 and 1992, respectively, and the Ph.D. degree in computer science from RMIT University, Melbourne, VIC, Australia, in 2000.

He is currently a Professor of Computer Science, the Head of the Evolutionary Computation Research Group, and an Associate Dean (Research and Innovation) with the Faculty of Engineering, Victoria University of Wellington, Wellington, New Zealand. His current research interests include evolutionary computation, particularly genetic programming and particle swarm optimization with application areas of image analysis, multiobjective optimization, feature selection and reduction, job-shop scheduling, and transfer learning.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 05,2026 at 08:08:09 UTC from IEEE Xplore. Restrictions apply.